

Next.js 14 App Router – Complete Theory Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 5 Oct 2026 – 11 Oct 2026 (Days 50–56)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Goal: Master Next.js 14 App Router for production MERN stacks — rendering strategies, server/client components, routing, data fetching, API routes, middleware, auth, SEO, and deployment.

Table of Contents

1. [Why Next.js?](#)
2. [App Router Architecture](#)
3. [Rendering Strategies: SSR, SSG, ISR, CSR](#)
4. [Server vs Client Components](#)
5. [Routing Deep Dive](#)
6. [Data Fetching Patterns](#)
7. [API Routes \(Route Handlers\)](#)
8. [Middleware](#)
9. [Authentication Patterns](#)
10. [SEO & Metadata](#)
11. [Performance Optimization](#)
12. [Deployment](#)
13. [Interview Q&A](#)

14. Practice Links & Resources

1. Why Next.js?

Next.js is a **React framework** that solves problems plain React cannot:

Problem (CRA/Vite SPA)	Next.js Solution
Poor SEO (empty HTML shell)	Server-rendered HTML with content
Slow first paint	Streaming SSR, automatic code splitting
No built-in routing	File-system App Router
Manual API setup	Route Handlers (<code>app/api/</code>)
Image optimization manual	<code><Image></code> component with lazy loading
No caching strategy	Built-in fetch cache, ISR, static generation

Companies using Next.js: Vercel, Netflix, TikTok, Nike, Uber, Adobe.

When to choose Next.js over Vite + React:

- SEO matters (marketing, blogs, e-commerce)
- You need SSR/SSG/ISR
- Full-stack in one repo (API + frontend)
- Deploying to Vercel or edge platforms

2. App Router Architecture

Next.js 14 uses the **App Router** (`app/` directory). The legacy **Pages Router** (`pages/`) still works but App Router is the default for new projects.

```

app/
├── layout.tsx           # Root layout (HTML shell, providers)
├── page.tsx             # Route: /
├── loading.tsx          # Suspense fallback for /
├── error.tsx            # Error boundary for /
├── not-found.tsx        # 404 page
├── globals.css
├── (marketing)/         # Route group – no URL segment
│   ├── layout.tsx
│   ├── about/page.tsx  # /about
│   └── pricing/page.tsx # /pricing
├── dashboard/
│   ├── layout.tsx      # Nested layout for /dashboard/*
│   ├── page.tsx        # /dashboard
│   ├── settings/page.tsx # /dashboard/settings
│   └── [teamId]/
│       └── page.tsx     # /dashboard/abc123 (dynamic)
├── blog/
│   └── [slug]/
│       └── page.tsx     # /blog/my-post
├── api/
│   └── users/
│       └── route.ts     # GET/POST /api/users
└── middleware.ts       # Edge middleware (root level)

```

Special Files Convention

File	Purpose
layout.tsx	Shared UI wrapper; persists across navigation
page.tsx	Unique UI for a route segment
loading.tsx	Instant loading UI via React Suspense
error.tsx	Client error boundary for segment
not-found.tsx	Custom 404 for segment
route.ts	API endpoint (no UI)
template.tsx	Re-mounts on navigation (unlike layout)
default.tsx	Parallel route fallback

Route Groups & Parallel Routes

```
// app/(auth)/login/page.tsx → URL is /login (group name hidden)
// app/(auth)/register/page.tsx → URL is /register

// Parallel routes – render multiple pages in one layout
// app/dashboard/@analytics/page.tsx
// app/dashboard/@team/page.tsx
// app/dashboard/layout.tsx receives { analytics, team } as props
```

3. Rendering Strategies: SSR, SSG, ISR, CSR

Understanding rendering is the #1 Next.js interview topic.

Comparison Table

Strategy	Full Name	When HTML is Built	Cache	Use Case
SSG	Static Site Generation	Build time	CDN forever	Blogs, docs, marketing
SSR	Server-Side Rendering	Every request	No cache (default)	Personalized dashboards
ISR	Incremental Static Regeneration	Build + revalidate	CDN + timed refresh	Product catalogs, news
CSR	Client-Side Rendering	Browser	None on server	Admin panels, auth-gated UI
Streaming	React Server Streaming	Request time, chunked	Partial	Large pages, slow data sources

SSG – Static Site Generation

```
// app/products/page.tsx
export default async function ProductsPage() {
  const products = await fetch("https://api.example.com/products", {
    cache: "force-cache", // default – static at build time
  }).then(r => r.json());

  return (
    <ul>
      {products.map((p: Product) => (
        <li key={p.id}>{p.name} – ${p.price}</li>
      ))}
    </ul>
  );
}
```

Dynamic SSG with `generateStaticParams`

```
// app/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const posts = await fetch("https://api.example.com/posts").then(r => r.json());
  return posts.map((post: Post) => ({ slug: post.slug }));
}

export default async function BlogPost({ params }: { params: { slug: string } }) {
  const post = await fetch(`https://api.example.com/posts/${params.slug}`, {
    cache: "force-cache",
  }).then(r => r.json());

  return <article><h1>{post.title}</h1><p>{post.body}</p></article>;
}
```

SSR – Server-Side Rendering

```
// app/dashboard/page.tsx
export const dynamic = "force-dynamic"; // opt out of static caching

export default async function Dashboard() {
  const session = await getSession(); // cookies read on every request
  const data = await fetch("https://api.example.com/user-stats", {
    cache: "no-store", // SSR – fresh data every request
    headers: { Authorization: `Bearer ${session.token}` },
  }).then(r => r.json());

  return <DashboardView stats={data} user={session.user} />;
}
```

ISR – Incremental Static Regeneration

```
// app/news/page.tsx
export default async function NewsPage() {
  const articles = await fetch("https://api.example.com/news", {
    next: { revalidate: 300 }, // regenerate every 5 minutes
  }).then(r => r.json());

  return <NewsList articles={articles} />;
}

// On-demand ISR – purge cache programmatically
// POST /api/revalidate?path=/news&secret=TOKEN
import { revalidatePath } from "next/cache";
revalidatePath("/news");
```

CSR – Client-Side Rendering

```
"use client";
import useSWR from "swr";

export default function LivePrices() {
  const { data, error, isLoading } = useSWR("/api/prices", fetcher, {
    refreshInterval: 5000,
  });

  if (isLoading) return <Spinner />;
  if (error) return <Error message={error.message} />;
  return <PriceTable prices={data} />;
}
```

Decision Flowchart (Mental Model)

```
Is content the same for all users?
├─ YES → Is it okay to be stale for N seconds?
│   ├── YES → ISR (revalidate: N)
│   └── NO → SSG with on-demand revalidation
└─ NO → Does it need SEO?
    ├── YES → SSR (cache: "no-store")
    └── NO → CSR (client component + SWR/React Query)
```

4. Server vs Client Components

Default in App Router: every component is a Server Component unless marked with `"use client"`.

Feature Comparison

Feature	Server Component	Client Component
Runs on	Server only	Server (SSR) + Browser
JavaScript sent to browser	None	Full component JS
<code>useState</code> , <code>useEffect</code>	✗	✓
Event handlers (<code>onClick</code>)	✗	✓
Browser APIs (<code>window</code> , <code>localStorage</code>)	✗	✓
Direct DB/API access	✓	✗ (use API routes)
<code>async/await</code> in component	✓	✗
Secrets in code	✓ (never exposed)	✗ (visible in bundle)
Context providers	✗ (can't use)	✓

Composition Pattern (Best Practice)

```
// app/page.tsx – Server Component
import { ClientCounter } from "../ClientCounter";
import { fetchProducts } from "@lib/db";

export default async function HomePage() {
  const products = await fetchProducts(); // server-only DB call

  return (
    <main>
      <h1>Products</h1>
      <ProductGrid products={products} />  {/* Server Component */}
      <ClientCounter />                      {/* Client island */}
    </main>
  );
}
```

```
// app/ClientCounter.tsx
"use client";
import { useState } from "react";

export function ClientCounter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(c => c + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Passing Data: Server → Client

```
// ✅ Serializable props only (JSON-safe)
<ClientChart data={chartData} userId={user.id} />

// ❌ Cannot pass functions, classes, or Date objects directly
<ClientChart onSelect={(id) => {}} /> // Error!
```

When to Use "use client"

Use Server Component	Use Client Component
Fetch data	Forms with controlled inputs
Access backend resources	Interactive UI (modals, dropdowns)
Keep sensitive info on server	Browser APIs
Reduce JS bundle size	Custom hooks
Static content display	Third-party libs needing browser

5. Routing Deep Dive

Dynamic Segments

```
// app/shop/[category]/[productId]/page.tsx
interface PageProps {
  params: { category: string; productId: string };
  searchParams: { sort?: string; page?: string };
}

export default function ProductPage({ params, searchParams }: PageProps) {
  // URL: /shop/electronics/abc123?sort=price&page=2
  return <div>{params.category} / {params.productId}</div>;
}
```

Catch-All & Optional Catch-All

```
// app/docs/[...slug]/page.tsx
// Matches: /docs/a, /docs/a/b, /docs/a/b/c
export default function DocsPage({ params }: { params: { slug: string[] } }) {
  return <div>Path: {params.slug.join("/")}</div>;
}

// app/shop/[...slug]/page.tsx – optional catch-all
// Matches: /shop AND /shop/a/b
```

Navigation APIs

```
"use client";
import Link from "next/link";
import { useRouter, usePathname, useSearchParams } from "next/navigation";

export function Nav() {
  const router = useRouter();
  const pathname = usePathname(); // "/dashboard"
  const searchParams = useSearchParams(); // "?tab=settings"

  return (
    <>
      <Link href="/about" prefetch={true}>About</Link>
      <button onClick={() => router.push("/dashboard?tab=settings")}>
        Go to Settings
      </button>
      <button onClick={() => router.back()}>Back</button>
    </>
  );
}
```

<Link> VS router.push

Method	Prefetch	Use When
<Link>	Yes (viewport)	Standard navigation
router.push()	No	Programmatic after action
router.replace()	No	Redirect without history entry
redirect() (server)	N/A	Server-side redirect

```
// Server-side redirect
import { redirect } from "next/navigation";

export default async function ProfilePage() {
  const session = await getSession();
  if (!session) redirect("/login");
  return <Profile user={session.user} />;
}
```

Route Handlers vs Pages

App Router	Pages Router (legacy)
<code>app/api/users/route.ts</code>	<code>pages/api/users.ts</code>
Named exports: <code>GET</code> , <code>POST</code>	Default export handler
Web Request/Response API	Node req/res objects

6. Data Fetching Patterns

Fetch Cache Options (Next.js 14)

```
// 1. Static (SSG) – cached indefinitely
fetch(url, { cache: "force-cache" });

// 2. Dynamic (SSR) – never cached
fetch(url, { cache: "no-store" });

// 3. ISR – cached with revalidation
fetch(url, { next: { revalidate: 60 } });

// 4. Tag-based revalidation
fetch(url, { next: { tags: ["products"] } });
// Later: revalidateTag("products");
```

Parallel Data Fetching

```
export default async function DashboardPage() {  
  // ❌ Sequential – slower  
  const user = await fetchUser();  
  const orders = await fetchOrders(user.id);  
  
  // ✅ Parallel – faster  
  const [user, orders, notifications] = await Promise.all([  
    fetchUser(),  
    fetchOrders(),  
    fetchNotifications(),  
  ]);  
  
  return <Dashboard user={user} orders={orders} notifications={notifications} />;  
}
```

React `cache()` for Request Deduplication

```
import { cache } from "react";  
  
export const getUser = cache(async (id: string) => {  
  const res = await fetch(`https://api.example.com/users/${id}`);  
  return res.json();  
});  
  
// Called multiple times in one request → only one fetch  
export default async function Page({ params }: { params: { id: string } }) {  
  const user = await getUser(params.id);  
  const posts = await getPostsByUser(params.id);  
  return <Profile user={user} posts={posts} />;  
}
```

Server Actions (Mutations)

```
// app/actions.ts
"use server";
import { revalidatePath } from "next/cache";
import { z } from "zod";

const CreatePostSchema = z.object({
  title: z.string().min(1).max(200),
  body: z.string().min(1),
});

export async function createPost(formData: FormData) {
  const parsed = CreatePostSchema.safeParse({
    title: formData.get("title"),
    body: formData.get("body"),
  });

  if (!parsed.success) {
    return { error: parsed.error.flatten() };
  }

  await db.post.create({ data: parsed.data });
  revalidatePath("/blog");
  return { success: true };
}
```

```
// app/blog/new/page.tsx
import { createPost } from "@/app/actions";

export default function NewPostPage() {
  return (
    <form action={createPost}>
      <input name="title" placeholder="Title" required />
      <textarea name="body" placeholder="Body" required />
      <button type="submit">Publish</button>
    </form>
  );
}
```

External Libraries (Prisma, Drizzle)

```
// lib/db.ts - server-only module
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma ?? new PrismaClient();

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

7. API Routes (Route Handlers)

```
// app/api/users/route.ts
import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";

const CreateUserSchema = z.object({
  name: z.string().min(1),
  email: z.string().email(),
});

export async function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams;
  const page = Number(searchParams.get("page") ?? "1");
  const limit = Number(searchParams.get("limit") ?? "10");

  const users = await prisma.user.findMany({
    skip: (page - 1) * limit,
    take: limit,
    select: { id: true, name: true, email: true },
  });

  return NextResponse.json({ users, page, limit });
}

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const parsed = CreateUserSchema.safeParse(body);

    if (!parsed.success) {
      return NextResponse.json(
        { error: "Validation failed", details: parsed.error.flatten() },
        { status: 400 }
      );
    }

    const user = await prisma.user.create({ data: parsed.data });
    return NextResponse.json(user, { status: 201 });
  } catch (error) {
    return NextResponse.json({ error: "Internal server error" }, { status: 500 });
  }
}
```

Dynamic API Routes

```
// app/api/users/[id]/route.ts
export async function GET(
  _request: NextRequest,
  { params }: { params: { id: string } }
) {
  const user = await prisma.user.findUnique({ where: { id: params.id } });
  if (!user) return NextResponse.json({ error: "Not found" }, { status: 404 });
  return NextResponse.json(user);
}

export async function DELETE(
  _request: NextRequest,
  { params }: { params: { id: string } }
) {
  await prisma.user.delete({ where: { id: params.id } });
  return new NextResponse(null, { status: 204 });
}
```

Route Handler Config

```
// Force dynamic – never cache this route
export const dynamic = "force-dynamic";
export const runtime = "nodejs"; // or "edge"
export const maxDuration = 30; // seconds (Vercel Pro)
```

8. Middleware

Middleware runs **before** a request completes — at the **Edge** runtime.

```
// middleware.ts (project root)
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import { verifyJWT } from "@lib/auth";

const PUBLIC_ROUTES = ["/", "/login", "/register", "/api/auth"];

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Skip public routes
  if (PUBLIC_ROUTES.some(route => pathname.startsWith(route))) {
    return NextResponse.next();
  }

  const token = request.cookies.get("session")?.value;

  if (!token) {
    const loginUrl = new URL("/login", request.url);
    loginUrl.searchParams.set("redirect", pathname);
    return NextResponse.redirect(loginUrl);
  }

  try {
    const payload = await verifyJWT(token);
    const response = NextResponse.next();
    response.headers.set("x-user-id", payload.sub);
    return response;
  } catch {
    return NextResponse.redirect(new URL("/login", request.url));
  }
}

export const config = {
  matcher: [
    "/dashboard/:path*",
    "/api/protected/:path*",
    "/((?!_next/static|_next/image|favicon.ico).*)",
  ],
};
```

Middleware Use Cases

Use Case	Implementation
Auth guard	Redirect unauthenticated users
A/B testing	Rewrite to variant URL
Geolocation	Redirect by country header
Rate limiting	Return 429 before hitting API
Bot protection	Block suspicious user agents
i18n	Rewrite <code>/en/about</code> → locale segment

Middleware Limitations

- Runs on Edge — no Node.js APIs (`fs` , native modules)
- Cannot read request body
- Keep logic lightweight (adds latency to every matched request)

9. Authentication Patterns

Pattern Comparison

Pattern	Library	Best For
NextAuth.js (Auth.js)	<code>@auth/core</code>	OAuth + credentials, sessions
Clerk	<code>@clerk/nextjs</code>	Managed auth, quick setup
Lucia	<code>lucia-auth</code>	Custom sessions, full control
JWT in httpOnly cookie	Custom	API-first, mobile clients

Auth.js (NextAuth v5) Example

```
// auth.ts
import NextAuth from "next-auth";
import GitHub from "next-auth/providers/github";
import Credentials from "next-auth/providers/credentials";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    GitHub,
    Credentials({
      credentials: { email: {}, password: {} },
      authorize: async (credentials) => {
        const user = await validateUser(credentials);
        return user ?? null;
      },
    }),
  ],
  callbacks: {
    jwt({ token, user }) {
      if (user) token.role = user.role;
      return token;
    },
    session({ session, token }) {
      session.user.role = token.role as string;
      return session;
    },
  },
});
```

```
// app/api/auth/[...nextauth]/route.ts
import { handlers } from "@auth";
export const { GET, POST } = handlers;
```

```
// Protect server component
import { auth } from "@auth";
import { redirect } from "next/navigation";

export default async function ProtectedPage() {
  const session = await auth();
  if (!session) redirect("/login");
  return <div>Hello {session.user.name}</div>;
}
```

Session Security Checklist

Rule	Why
Store JWT in <code>httpOnly</code> cookie	Prevents XSS token theft
Set <code>Secure</code> flag in production	HTTPS only
Set <code>SameSite=Lax</code> or <code>Strict</code>	CSRF protection
Short expiry + refresh token	Limits damage if stolen
Never store tokens in localStorage	Accessible to any JS

10. SEO & Metadata

Static Metadata

```
// app/about/page.tsx
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "About Us | MyApp",
  description: "Learn about our mission and team.",
  openGraph: {
    title: "About Us",
    description: "Learn about our mission.",
    url: "https://myapp.com/about",
    siteName: "MyApp",
    images: [{ url: "https://myapp.com/og-about.png", width: 1200, height: 630 }],
    locale: "en_US",
    type: "website",
  },
  twitter: {
    card: "summary_large_image",
    title: "About Us",
    description: "Learn about our mission.",
    images: ["https://myapp.com/og-about.png"],
  },
  robots: { index: true, follow: true },
};
```

Dynamic Metadata

```
// app/blog/[slug]/page.tsx
export async function generateMetadata(
  { params }: { params: { slug: string } }
): Promise<Metadata> {
  const post = await fetchPost(params.slug);

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      publishedTime: post.publishedAt,
      authors: [post.author.name],
    },
  };
}
```

Structured Data (JSON-LD)

```
export default async function ProductPage({ params }: { params: { id: string } }) {
  const product = await fetchProduct(params.id);

  const jsonLd = {
    "@context": "https://schema.org",
    "@type": "Product",
    name: product.name,
    description: product.description,
    offers: {
      "@type": "Offer",
      price: product.price,
      priceCurrency: "USD",
    },
  };

  return (
    <>
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd) }}
    />
    <ProductDetail product={product} />
    </>
  );
}
```

SEO Checklist

Item	Implementation
Unique <code><title></code> per page	<code>metadata.title</code>
Meta description	<code>metadata.description</code>
Canonical URL	<code>metadata.alternates.canonical</code>
Open Graph tags	<code>metadata.openGraph</code>
Sitemap	<code>app/sitemap.ts</code>
Robots.txt	<code>app/robots.ts</code>
Semantic HTML	<code><main></code> , <code><article></code> , <code><h1></code> hierarchy
Core Web Vitals	Image optimization, font loading

```
// app/sitemap.ts
import { MetadataRoute } from "next";

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const posts = await fetchAllPosts();
  return [
    { url: "https://myapp.com", lastModified: new Date(), changeFrequency: "weekly",
    ...posts.map(post => ({
      url: `https://myapp.com/blog/${post.slug}`,
      lastModified: new Date(post.updatedAt),
      changeFrequency: "monthly" as const,
      priority: 0.8,
    })),
  ];
}
```

11. Performance Optimization

Image Optimization

```
import Image from "next/image";

<Image
  src="/hero.jpg"
  alt="Hero banner"
  width={1200}
  height={600}
  priority                // LCP image - preload
  placeholder="blur"
  blurDataURL={blurDataUrl}
  sizes="(max-width: 768px) 100vw, 50vw"
/>
```

Font Optimization

```
// app/layout.tsx
import { Inter, JetBrains_Mono } from "next/font/google";

const inter = Inter({ subsets: ["latin"], variable: "--font-inter" });
const mono = JetBrains_Mono({ subsets: ["latin"], variable: "--font-mono" });

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={` ${inter.variable} ${mono.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

Streaming with Suspense

```
import { Suspense } from "react";

export default function Page() {
  return (
    <main>
      <Hero /> {/* renders immediately */}
      <Suspense fallback={<Skeleton />}>
        <SlowDataComponent /> {/* streams when ready */}
      </Suspense>
    </main>
  );
}
```

Bundle Analysis

```
# Analyze bundle size
npm install @next/bundle-analyzer
# next.config.js
const withBundleAnalyzer = require("@next/bundle-analyzer")({ enabled: process.env
module.exports = withBundleAnalyzer({});
```

Optimization	Impact
Server Components	Smaller client bundle
Dynamic imports	Code-split heavy libs
<code>next/image</code>	Automatic WebP/AVIF
<code>next/font</code>	Zero layout shift
Streaming SSR	Faster TTFB
Route prefetching	Instant navigation

12. Deployment

Vercel (Recommended)

```
# Install Vercel CLI
npm i -g vercel

# Deploy
vercel          # preview
vercel --prod   # production
```

Environment Variables

```
# .env.local (never commit)
DATABASE_URL=postgresql://...
NEXTAUTH_SECRET=...
NEXTAUTH_URL=http://localhost:3000

# Vercel dashboard → Settings → Environment Variables
# Set for: Production, Preview, Development
```

next.config.js Production Settings

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: "https", hostname: "cdn.example.com" },
    ],
  },
  experimental: {
    serverActions: { bodySizeLimit: "2mb" },
  },
  headers: async () => [
    {
      source: "/(.*)",
      headers: [
        { key: "X-Frame-Options", value: "DENY" },
        { key: "X-Content-Type-Options", value: "nosniff" },
      ],
    },
  ],
};

module.exports = nextConfig;
```

Deployment Platform Comparison

Platform	SSR	Edge	Cost	Best For
Vercel	✓	✓	Free tier	Next.js native
Netlify	✓	✓	Free tier	JAMstack
AWS Amplify	✓	✗	Pay-as-go	AWS ecosystem
Docker + VPS	✓	✗	Fixed cost	Full control
Railway	✓	✗	Usage-based	Simple backend

Docker Deployment

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

13. Interview Q&A

Q1: What is the difference between SSR, SSG, and ISR?

Answer: SSG builds HTML at **build time** — fastest, cached on CDN, same for all users. SSR builds HTML on **every request** — fresh, personalized, slower. ISR is SSG with a **revalidation timer** — static speed with periodic updates. Choose SSG for docs, SSR for user dashboards, ISR for product pages that change occasionally.

Q2: When do you use Server vs Client Components?

Answer: Default to Server Components for data fetching, SEO content, and zero-JS UI. Use Client Components only when you need interactivity (`useState` , event handlers), browser APIs, or third-party client libraries. Compose by keeping client islands small inside server layouts.

Q3: How does Next.js caching work in App Router?

Answer: Next.js 14 has four cache layers: (1) Request memoization — deduplicates fetch in one render, (2) Data Cache — stores fetch results across requests, (3) Full Route Cache — caches rendered HTML/RSC payload, (4) Router Cache — client-side cache of visited routes. Control with `cache`, `revalidate`, and `revalidatePath/Tag`.

Q4: What is middleware and when would you use it?

Answer: Middleware runs at the Edge before route handlers. Use it for auth checks, redirects, A/B tests, geolocation routing, and rate limiting. Keep it stateless and fast — it runs on every matched request. Cannot access DB directly; use JWT verification or edge-compatible stores.

Q5: How do Server Actions differ from API routes?

Answer: Server Actions are `"use server"` functions called directly from forms or event handlers — no fetch boilerplate, automatic CSRF protection, progressive enhancement. API routes are REST endpoints for external clients, webhooks, or mobile apps. Use Server Actions for internal mutations; API routes for public/external APIs.

Q6: How do you optimize Core Web Vitals in Next.js?

Answer: LCP — use `priority` on hero images, server-render above-fold content. CLS — use `next/font`, explicit image dimensions. INP — minimize client JS, use Server Components, code-split heavy components. Also: streaming SSR, prefetching with `<Link>`, and ISR for static content.

Q7: Explain the App Router file conventions.

Answer: `layout.tsx` wraps child routes and persists state. `page.tsx` defines the UI for a route. `loading.tsx` provides Suspense fallback. `error.tsx` is a client error boundary. `route.ts` defines API endpoints. `generateStaticParams` pre-builds dynamic routes. Route groups (`folder`) organize without affecting URL.

Q8: How do you handle authentication in Next.js 14?

Answer: Use Auth.js for OAuth + sessions, or JWT in httpOnly cookies for custom auth. Protect routes via middleware (redirect) or server component checks (`auth()` + `redirect()`). Never store tokens in localStorage. Use Server Actions or API routes for login/logout. Pass session data to Client Components as serializable props only.

14. Practice Links & Resources

Official Documentation

- [Next.js 14 Docs](#) — App Router reference
- [Next.js Learn Course](#) — Interactive tutorial
- [React Server Components](#) — RSC spec

Practice Projects

Project	Skills Covered
Blog with MDX + ISR	SSG, dynamic routes, metadata
E-commerce storefront	ISR, Server Actions, cart state
Dashboard with auth	SSR, middleware, protected routes
URL shortener	API routes, edge middleware
Real-time chat	Client components, WebSockets

Tools

- [Vercel Templates](#) — Starter projects
- [shadcn/ui](#) — Copy-paste components for Next.js
- [T3 Stack](#) — Next.js + tRPC + Prisma + Tailwind

Interview Prep

- [Next.js GitHub Discussions](#) — Architecture decisions
- [Patterns.dev](#) — Rendering patterns
- Build and deploy 2–3 full Next.js projects before interviews

Next file: [05-nodejs-express-complete.md](#)

Product Company Interview Bank – Next.js (50 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart **Rule:** Full terms only — spell out JWT as JSON Web Token, API as Application Programming Interface, etc.

Question 1: What is Next.js and why use it over plain React?

Answer: Next.js is a React framework created by Vercel that adds production features on top of React: file-based routing, server-side rendering, static site generation, built-in Application Programming Interface (API) routes, image optimization, and more. Companies use it because it solves common web app problems (Search Engine Optimization, performance, deployment) without building everything from scratch. It powers sites at Netflix, TikTok, and many startups.

Question 2: What is the difference between the Pages Router and App Router?

Answer: The Pages Router (Next.js 12 and earlier) uses a `pages/` directory — each file becomes a route, and data fetching uses `getServerSideProps`, `getStaticProps`, or `getInitialProps`. The App Router (Next.js 13+) uses an `app/` directory with nested layouts, React Server Components by default, and simpler data fetching with async server components. New projects should use the App Router; Pages Router is still supported for existing apps.

Question 3: What is Server-Side Rendering (SSR)?

Answer: Server-Side Rendering means the server generates HTML for each request by running React on the server, then sends complete HTML to the browser. The page shows content quickly

and is crawlable by search engines. In the App Router, a page is SSR when it is dynamic (not statically cached) — for example, using `cookies()`, `headers()`, or `export const dynamic = 'force-dynamic'`. After HTML arrives, React hydrates the page for interactivity.

Question 4: What is Static Site Generation (SSG)?

Answer: Static Site Generation pre-builds HTML at build time for pages that do not change per user. The HTML is served from a Content Delivery Network (CDN), making responses very fast and cheap. In the App Router, pages are static by default unless they use dynamic functions or opt out. Use SSG for marketing pages, blogs, and documentation where content is the same for everyone.

Question 5: What is Incremental Static Regeneration (ISR)?

Answer: Incremental Static Regeneration lets you update static pages after build without rebuilding the entire site. You set a revalidation time (e.g., 60 seconds). The first request after that window triggers a background rebuild; subsequent requests get the fresh page. In the App Router, use `export const revalidate = 60` or `next: { revalidate: 60 }` in fetch options. Ideal for product catalogs or news that changes occasionally.

Question 6: What are React Server Components in Next.js?

Answer: React Server Components (RSC) run only on the server. They can fetch data, read files, and query databases directly without exposing credentials to the browser. They do not ship JavaScript to the client for their own logic. In the App Router, components in `app/` are Server Components by default. They cannot use `useState`, `useEffect`, or browser APIs. Add `'use client'` at the top to make a Client Component.

Question 7: What are Client Components in Next.js?

Answer: Client Components run in the browser and support interactivity — clicks, forms, hooks, and browser APIs. Mark them with `'use client'` at the top of the file. Use Client Components for: event handlers, `useState` / `useEffect`, custom hooks, and third-party libraries that use browser APIs. Keep Client Components as small as possible ("leaf" pattern) — wrap interactive parts, not entire pages.

Question 8: How does file-based routing work in the App Router?

Answer: Folders define route segments; `page.tsx` makes a route publicly accessible; `layout.tsx` wraps shared user interface; `loading.tsx` shows while content loads; `error.tsx` handles errors; `not-found.tsx` handles 404s. Example: `app/blog/[slug]/page.tsx` maps to `/blog/my-post`. Route groups `(marketing)` organize files without affecting the URL. Parallel routes `@modal` and intercepting routes enable advanced patterns like modals over pages.

Question 9: What are layouts in Next.js?

Answer: Layouts wrap child pages and persist across navigation without re-rendering — ideal for navbars, sidebars, and footers. In `app/dashboard/layout.tsx`, the layout wraps all routes under `/dashboard`. Layouts can nest: root layout → dashboard layout → page. Unlike Pages Router `_app.js`, App Router layouts do not remount when navigating between sibling routes, preserving state in the layout.

Question 10: What is the root layout?

Answer: `app/layout.tsx` is the root layout required in every App Router project. It must include `<html>` and `<body>` tags. It wraps all pages and is the place for global styles, fonts, metadata defaults, and providers. Only one root layout exists. Changes here affect the entire application.

Question 11: How do you fetch data in Server Components?

Answer: Call `fetch` or database clients directly inside async Server Components — no `useEffect` needed. Example: `async function Page() { const res = await fetch('https://api.example.com/posts'); const posts = await res.json(); return <ul>...</ul> }`. Next.js extends `fetch` with caching options: `{ cache: 'force-cache' }` for static, `{ cache: 'no-store' }` for dynamic, `{ next: { revalidate: 3600 } }` for Incremental Static Regeneration.

Question 12: What is the difference between `cache: 'no-store'` and `revalidate` ?

Answer: `cache: 'no-store'` fetches fresh data on every request — fully dynamic, like Server-Side Rendering on each visit. `next: { revalidate: N }` caches the result for N seconds, then regenerates in the background (Incremental Static Regeneration). Choose `no-store` for personalized or real-time data (user dashboard). Choose `revalidate` for semi-static content (product list updated hourly).

Question 13: What is Next.js Middleware?

Answer: Middleware runs before a request completes — at the edge, close to the user. Define it in `middleware.ts` at the project root. Use cases: authentication redirects, geolocation-based routing, A/B testing, rewriting URLs, and setting headers. It runs on the Edge Runtime (limited Node.js APIs). Example: redirect unauthenticated users from `/dashboard` to `/login`.

Question 14: How do you handle authentication in Next.js?

Answer: Common approaches: (1) Middleware checks session or JSON Web Token (JWT) cookie and redirects. (2) Server Components read session from cookies via `cookies()` and conditionally render. (3) Libraries like NextAuth.js (Auth.js) handle OAuth, sessions, and callbacks. (4) Client-side auth for Single Page Application-style flows with protected Client Components. Prefer server-side session checks for security-sensitive routes.

Question 15: What are API Routes in Next.js?

Answer: API Routes let you build backend endpoints inside your Next.js app. In the App Router, create `app/api/users/route.ts` with exported `GET`, `POST`, `PUT`, `DELETE` functions. In the Pages Router, use `pages/api/users.ts`. They run on the server and are useful for form handling, webhooks, and Backend-for-Frontend patterns without a separate Express server.

Question 16: What is the Backend-for-Frontend (BFF) pattern in Next.js?

Answer: Backend-for-Frontend means your Next.js API routes act as a thin layer between the frontend and external services. They hide API keys, aggregate multiple backend calls into one response, and shape data for the user interface. Example: `/api/dashboard` fetches user, orders, and notifications server-side and returns one JSON object — faster and more secure than exposing multiple keys to the browser.

Question 17: What is `next/image` and why use it?

Answer: `next/image` is Next.js's optimized Image component. It automatically serves correctly sized images, lazy-loads below-the-fold images, supports modern formats like WebP, and prevents layout shift with width/height. It can optimize remote images when you configure `remotePatterns` in `next.config.js`. Using it improves Largest Contentful Paint (LCP) — a Core Web Vital that affects Search Engine Optimization and user experience.

Question 18: What is `next/font` ?

Answer: `next/font` loads Google Fonts or local fonts at build time and self-hosts them — no external requests to Google at runtime. It eliminates layout shift by computing font metrics and applies `font-display: swap` automatically. Example: `import { Inter } from 'next/font/google'` then apply `inter.className` to your layout. Improves performance and privacy.

Question 19: What is `next/link` ?

Answer: `next/link` is the Next.js navigation component for client-side transitions between pages without full page reloads. It prefetches linked pages in the viewport by default (in production), making navigation feel instant. Use `<Link href="/about">About</Link>` instead of `<a>` for internal routes. For external links, use regular `<a>` with `target="_blank"` and `rel="noopener noreferrer"`.

Question 20: What is prefetching in Next.js?

Answer: Prefetching loads page data and JavaScript in the background before the user clicks a link. `<Link>` prefetches routes in the viewport automatically. You can disable with `prefetch={false}`. In the App Router, static routes prefetch fully; dynamic routes prefetch partially until visited. Prefetching makes Single Page Application-style navigation feel instant while keeping Search Engine Optimization benefits.

Question 21: What are loading and streaming in the App Router?

Answer: `loading.tsx` automatically wraps a page in a Suspense boundary — show a skeleton while the page loads. React streams HTML to the browser as Server Components resolve, so users see the shell immediately and content fills in progressively. Nested `loading.tsx` files give granular loading states per route segment. This improves perceived performance on slow data fetches.

Question 22: What is `Suspense` in Next.js?

Answer: Suspense lets you defer rendering part of the page until data is ready, showing a fallback meanwhile. In the App Router, wrap slow Server Components or use `loading.tsx` for automatic Suspense. You can also wrap Client Components that lazy-load. Streaming sends the fallback HTML first, then replaces it when ready — users see progress instead of a blank screen.

Question 23: What is `error.tsx` in Next.js?

Answer: `error.tsx` is an error boundary for a route segment. When an error is thrown during rendering or data fetching, Next.js shows this component instead of crashing the whole app. It must be a Client Component (`'use client'`) because error boundaries require client-side React. It receives `error` and `reset` props — `reset()` tries re-rendering the segment. Place at appropriate levels for graceful degradation.

Question 24: What is `not-found.tsx` ?

Answer: `not-found.tsx` renders when you call the `notFound()` function from `next/navigation` — for example, when a blog post slug does not exist in the database. It returns a 404 status. Each route segment can have its own `not-found.tsx` . The root `app/not-found.tsx` handles global 404s for unmatched routes.

Question 25: What are dynamic route segments?

Answer: Dynamic segments use square brackets in folder names: `[id]` , `[slug]` , or catch-all `[...slug]` . Access params in Server Components via `async function Page({ params }: { params: { id: string } })` . Optional catch-all `[...slug]` matches zero or more segments. Use for user profiles (`/users/[id]`), blog posts (`/blog/[slug]`), and documentation paths.

Question 26: What are Route Handlers vs Server Actions?

Answer: Route Handlers (`app/api/.../route.ts`) are REST-style endpoints — good for webhooks, third-party integrations, and clients that expect HTTP APIs. Server Actions are async functions marked with `'use server'` called directly from forms and components — good for mutations like creating a post or updating a profile. Server Actions reduce boilerplate; Route Handlers offer full HTTP control (status codes, headers).

Question 27: What are Server Actions?

Answer: Server Actions are functions that run on the server, invoked from Client or Server Components — typically via forms. Mark a file or function with `'use server'`. Example: `<form action={createPost}>`. They integrate with React 19 Actions for pending states. Benefits: no separate API route, automatic POST requests, progressive enhancement (forms work without JavaScript), and secrets stay on the server.

Question 28: What is `useRouter` from `next/navigation` ?

Answer: In the App Router, `useRouter` from `next/navigation` (not `next/router`) provides programmatic navigation: `router.push('/dashboard')`, `router.replace('/login')`, `router.back()`, and `router.refresh()` to re-fetch server data. It is a Client Component hook. Use it after form success, logout, or conditional redirects based on client state.

Question 29: What is the difference between `router.push` and `router.replace` ?

Answer: `router.push('/path')` navigates to a new URL and adds an entry to the browser history — the back button returns to the previous page. `router.replace('/path')` navigates without adding history — useful after login (user should not "back" into the login page) or redirect chains. Both trigger client-side navigation in Next.js.

Question 30: What is `generateStaticParams` ?

Answer: `generateStaticParams` pre-generates static pages for dynamic routes at build time. In `app/blog/[slug]/page.tsx`, export `async function generateStaticParams() { return [{ slug: 'post-1' }, { slug: 'post-2' }] }`. Next.js builds those paths statically. Combine with `revalidate` for Incremental Static Regeneration on paths not pre-built. Essential for static blogs and product pages with known slugs.

Question 31: What is `generateMetadata` ?

Answer: `generateMetadata` exports dynamic metadata (title, description, Open Graph tags) for Search Engine Optimization. It runs on the server and can fetch data: `export async function generateMetadata({ params }) { const post = await getPost(params.slug); return { title: post.title, description: post.excerpt } }`. Replaces manual `<Head>` usage from the Pages Router. Supports static and dynamic metadata per route.

Question 32: How does Next.js help with Search Engine Optimization (SEO)?

Answer: Server-rendered HTML is fully crawlable by search engines. `generateMetadata` sets titles, descriptions, and social cards. Static generation gives fast page loads (ranking factor). Automatic sitemaps via `app/sitemap.ts`. Structured data in Server Components. Image optimization improves Core Web Vitals. Compared to client-only React, Next.js pages index reliably and load faster.

Question 33: What are Core Web Vitals and how does Next.js improve them?

Answer: Core Web Vitals are Google's user experience metrics: Largest Contentful Paint (LCP — loading speed), Interaction to Next Paint (INP — responsiveness), Cumulative Layout Shift (CLS — visual stability). Next.js helps via: `next/image` and `next/font` for LCP/CLS, Server Components for smaller JavaScript bundles (faster INP), streaming for early content, and automatic code splitting per route.

Question 34: What is the `next.config.js` file?

Answer: `next.config.js` (or `.mjs` / `.ts`) configures Next.js behavior: redirects, rewrites, headers, image domains, environment variables exposed to the browser, experimental features, and output mode (`standalone` for Docker). Example: allow images from `images.remotePatterns`, set `basePath` for deploying under a subpath, or enable `reactStrictMode` for development checks.

Question 35: What is the `standalone` output mode?

Answer: `output: 'standalone'` in `next.config.js` creates a minimal self-contained build in `.next/standalone` with only necessary files for production — ideal for Docker containers. It traces dependencies and copies only what is needed, reducing image size compared to copying all of `node_modules`. Pair with a multi-stage Dockerfile for efficient deployments.

Question 36: How do you deploy a Next.js app?

Answer: Common options: (1) Vercel — zero-config, built by Next.js creators, edge functions and analytics included. (2) Docker on AWS, Google Cloud, or Azure — use `standalone` output. (3) Node.js server — `next build && next start`. (4) Static export — `output: 'export'` for fully static sites on any static host (no Server-Side Rendering). Choose based on need for server features vs static hosting cost.

Question 37: What is Partial Prerendering (PPR)?

Answer: Partial Prerendering (experimental) combines static and dynamic content on the same page. The static shell (header, layout) is pre-rendered at build time; dynamic holes (personalized content) stream in at request time. You wrap dynamic parts in `<Suspense>`. Goal: static speed with dynamic personalization — best of Static Site Generation and Server-Side Rendering on one page.

Question 38: What are parallel routes in Next.js?

Answer: Parallel routes use the `@folder` convention to render multiple pages in the same layout simultaneously — e.g., `@analytics` and `@team` slots in a dashboard. Each slot has its own `loading.tsx` and `error.tsx`. Useful for split views, modals (`@modal`), and independent loading of dashboard panels. Defined in the layout: `{ children, analytics, team }`.

Question 39: What are intercepting routes?

Answer: Intercepting routes load a route inside the current layout while keeping the URL — common for photo gallery modals. Use `(.)` to intercept at the same level, `(..)` one level up. Example: clicking a photo on `/feed` opens `/photo/1` as a modal overlay; refreshing shows the full page. Combines with parallel routes `@modal` for polished user experience patterns.

Question 40: What is the Edge Runtime vs Node.js Runtime?

Answer: Next.js supports two runtimes. Node.js Runtime (default) — full Node.js APIs, suitable for database connections and heavy logic. Edge Runtime — lightweight V8 isolates at CDN edge locations, faster cold starts, limited APIs (no native Node modules), ideal for Middleware and low-latency redirects. Choose Edge for auth checks and geolocation; Node.js for complex server logic.

Question 41: How do environment variables work in Next.js?

Answer: Store secrets in `.env.local` (gitignored). Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser; all others are server-only. Access via `process.env.MY_SECRET` in Server Components and API routes. Never put database passwords or API keys in `NEXT_PUBLIC_` variables. Use `.env.development`, `.env.production` for environment-specific values.

Question 42: What is `next/headers` and `next/cookies` ?

Answer: `headers()` and `cookies()` from `next/headers` read request headers and cookies in Server Components and Server Actions. They are async in Next.js 15+. Using them opts the route into dynamic rendering because values vary per request. Example: read auth token from cookies to personalize a dashboard. These replace accessing `req.headers` from Pages Router `getServerSideProps`.

Question 43: What changed in Next.js 15?

Answer: Next.js 15 (2024) highlights: `fetch` requests no longer cached by default (dynamic by default), async request APIs (`cookies()` , `headers()` , `params` , `searchParams`), Turbopack stable for development, improved caching semantics with `staleTimes` , React 19 support, and `@next/codemod` for upgrading. Understand default caching changes when migrating from Next.js 14.

Question 44: What is Turbopack?

Answer: Turbopack is Vercel's Rust-based bundler, successor to Webpack for development. It offers faster local development startup and Hot Module Replacement (HMR) — especially in large codebases. Enable with `next dev --turbopack` . Production builds still use Webpack by default (Turbopack for production is evolving). It is the default dev bundler direction for Next.js.

Question 45: How do you handle internationalization (i18n) in the App Router?

Answer: App Router i18n is often implemented manually: use a `[locale]` dynamic segment (`app/[locale]/page.tsx`), middleware to detect locale from headers or cookies, and a dictionary file for translations. Libraries like `next-intl` simplify this. Unlike Pages Router built-in i18n config, App Router gives more flexibility but requires setup. Middleware redirects `/` to `/en` or `/hi` based on preference.

Question 46: What is the difference between redirect and rewrite in `next.config.js` ?

Answer: A redirect sends the browser to a new URL (user sees URL change) — use for moved pages or www to non-www. A rewrite serves content from another path internally (URL stays the same) — use for proxying APIs or friendly URLs. Redirect: `{ source: '/old', destination: '/new', permanent: true }` . Rewrite: `{ source: '/api/:path*', destination: 'https://backend.com/:path*' }` .

Question 47: How do you test a Next.js application?

Answer: Unit test components with Jest/Vitest and React Testing Library. Test API routes by calling handler functions directly. End-to-end tests with Playwright or Cypress against `next dev` or production build. Test Server Components by testing the async function output. Mock `fetch` and database calls. Next.js docs recommend Playwright for integration tests including navigation and server rendering.

Question 48: What are common Next.js performance mistakes?

Answer: (1) Marking entire pages `'use client'` when only a button needs interactivity. (2) Fetching in Client Components with `useEffect` instead of Server Components. (3) Not using `next/image` for large images. (4) Disabling prefetch unnecessarily. (5) Large client bundles from importing heavy libraries in Client Components. (6) Not using `loading.tsx` — users see blank screens. Fix by keeping Server Components default and profiling with Lighthouse.

Question 49: Pages Router: explain `getServerSideProps` vs `getStaticProps` ?

Answer: Still asked for legacy codebases. `getStaticProps` runs at build time — generates static HTML (Static Site Generation). `getServerSideProps` runs on every request — fresh data (Server-Side Rendering). `getStaticProps` with `revalidate` enables Incremental Static Regeneration. In App Router, these map to default static fetch vs `cache: 'no-store'` vs `revalidate` — but know the Pages Router names for interviews on older projects.

Question 50: How would you architect a MERN-style app with Next.js?

Answer: Use Next.js as the full stack: React Server Components for data-heavy pages fetching from MongoDB directly or via internal API; Client Components for interactive forms and dashboards; Route Handlers or Server Actions for mutations; Middleware for auth; `next/image` for media; deploy on Vercel or Docker. MongoDB connection pooled in a shared

`lib/mongodb.ts` module. Separate Express only if you need WebSockets or microservices — Next.js often replaces Express for standard REST apps.